

DataBase Management System

Data - set of raw bits/bytes.

Information - processed data used for decision making.

Database - an electronic system where data can be easily stored, accessed and modified.

DBMS - collection of inter related data + set of operations for handling data conveniently and efficiently.

Problem with file system:

- Data Redundancy and inconsistency.
- Difficulty in data access.
- Data isolation.
- Integrity problem.
- Atomicity problems.
- Concurrent access anomalies.
- Security of data.

Three level architecture:

DBMS maintains data in form of abstract views. For example if there are multiple departments of some company accessing data they must get data that is necessary for them and not all tables of database.

1. Physical Level / Internal Level abstraction:

- lowest level, describes how data is stored, low level data structures (RB, B+, Nary trees).
- Efficient algorithms for data compression and decompression and physical schema are required.
- Goal is fast and efficient access.

2. Logical Level / Conceptual Level abstraction:

- the physical data is mapped to conceptual schema. (Physical disk/file storage -> tables of database).
- This level describes about what data is stored and what relationships are there between tables.
- Goal is ease of use.

3. View Level / External Level:

- highest level of abstraction, describes views to show to the end users interacting with the app / database.
- manages the external level subschemas for different views, maintains security by hiding some data.
- Goal is to let end user interact.

Any real time snapshot of database is the instance of DB at that time.

Logical Schema is known as DB schema only.

Data Models are used to describe this logical schema consisting: data, relationships, semantics and constraints.

eg: ER Model, Relational Model, Object oriented model, Object-relational model.

DB Schema involves:

1. Attributes of a table.
2. Consistency constraints.
3. Relationships.

Languages needed for interacting with DBMS:

1. Data Definition Language (DDL): to specify database schema. Ensures constraints every time DB updates.
2. Data Manipulation Language (DML): to express queries and updates of DB.

There is a single language that has both of these languages know as SQL.

How does our application program understand SQL:

There are interfaces in all languages that help SQL and native application program to interact.

java - *"jdbc" - open database connectivity*

c/cpp - *"odbc" - java database connectivity*

Database Administrator (DBA):

Central control over data and functions relation to that data.

example functionalities of DBA are as follows:

1. schema definition
2. structure and access methods on database
3. physical and conceptual schema modifications
4. authorisation control over different API's
5. Maintenance like backups, security checks and upgrade in database.

DBMS Application Architecture:

Based on where client machine and server machines are present.

1. Tier 1 (T1) - single component app, client, server, DB on same machine
2. Tier 2 (T2) - 2 components app, client sends SQL through interfaces and DB runs on server of other machine.
3. Tier 3 (T3) - 3 component app, client is just frontend, client -> app server -> DB server.

Data Models:

Collection of conceptual tools for describing data, describing relationships, data semantics and consistency constraints.

ER Model:

Based on objects called entities and their relationships with each other. It's graphical mapping is known as ER Diagram which acts as blue print for DB schema construction.

Entity - thing or object of real world, independent of everything. eg Bhavyam, Divyam (both are students)

Strong Entity - Uniquely identified.

Weak Entity - Depends on some other Strong Entity for uniqueness.

Entity Set - set of all entities that share same properties but have individual existence. eg: Students

Attributes - an entity is represented by set of attributes. For each attribute there is a permitted set of values.

Types of Attribute:

Simple - can't be divided further. eg. bank account number.

Composite - can be divided into further parts. eg. name, address.

Single-valued - only one value. eg. student id, loan number

Multi-valued - having multiple values (can define how much limit). eg. nominees, phone number.

Derived - made from other attribute's values. eg. age from DOB, payable amount from interest and date of issue.

Relationships between Entity sets:

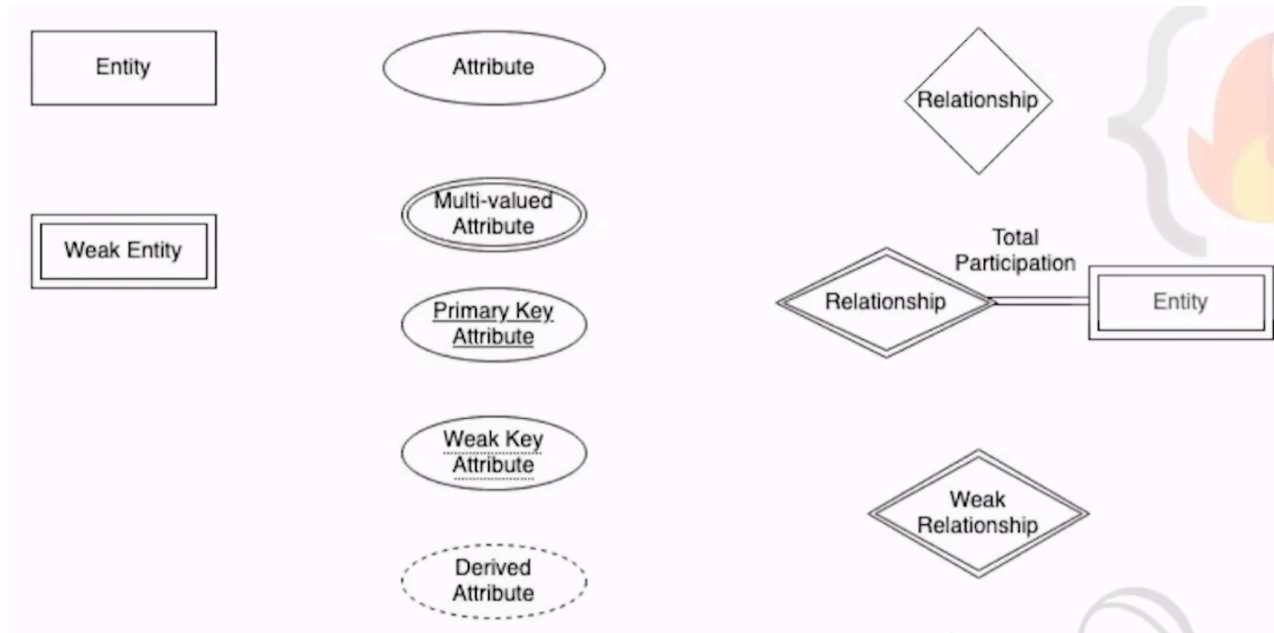
1:1 - one entity in entity set A linked to one entity in entity set B. eg. person to identity card.

1:N - one to many. eg. citizen to vehicles.

M:1 - many to one. eg. Courses taken by prof.

M:N - many to many. eg. customer and product.

Symbols to represent relationships:



Participation Constraints:

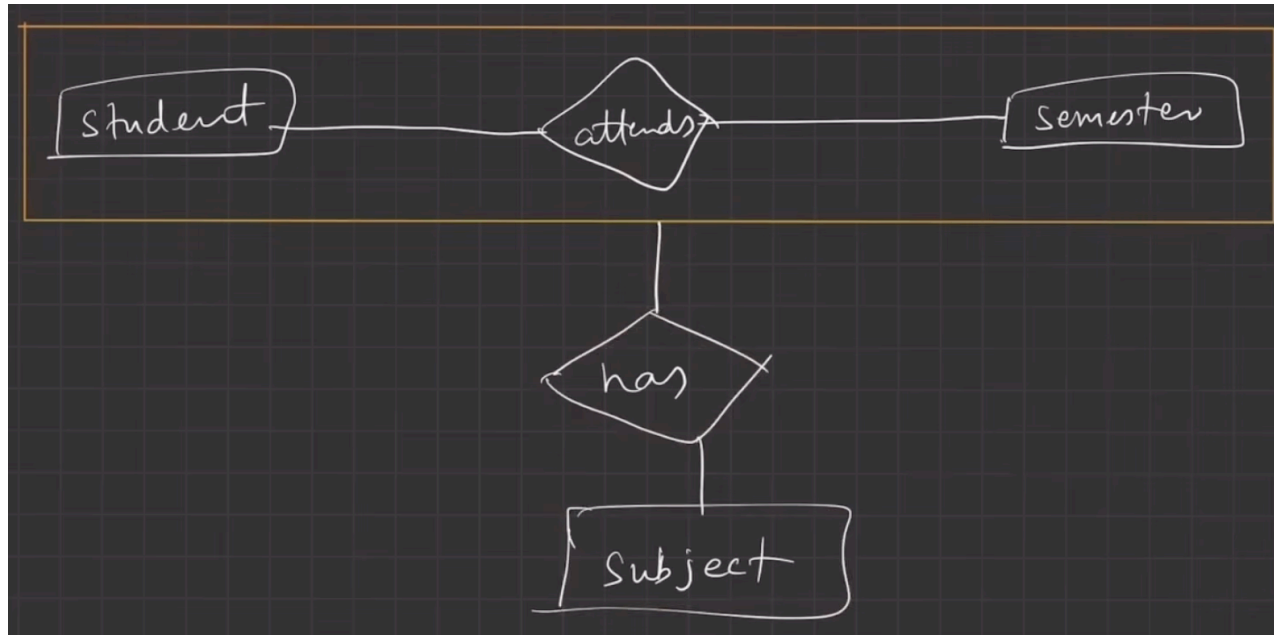
Total Participation - every entity in set A is linked to someone in set B.

Partial Participation - every entity in set A is not linked to someone in set B.

Weak entity set always have total participation.

Extended features of ER Model:

1. Specialisation - "is-a" relationship between entity sets to remove redundancy of data. This is a top down approach as we break the bigger entity set into smaller entity set which posses some different features. eg. people can be broken down to customers, employees and employees further into HR, SDE, BA, ML Engineer etc. This is used to remove redundancy and separation of functions of new entities made.
2. Generalisation - "is-a" relationship Top down approach, if we have some common attribute is some smaller entities that are a part of some bigger entity sets we can store that attribute in bigger entity set only. eg. bigger entity set is car and smaller are electric and petrol cars. So date of manufacture should be attribute of car entity set.
3. Participation Inheritance - If a parent entity set participates in some relationship then child entity sets also participate in that relationship.
4. Aggregation - Used to show relationships (new relationship) between relationships (some old relationships). This is made using abstraction and creating the old relationship a higher level entity (known as abstract entity). This too avoids data redundancy and hides complexity of understanding nested relationships. eg.



How to make ER Diagram ?

1. Identify Entities / Entity Set.
2. Identify Attributes for each entity and consistency constraints as well.
3. Identify relationships between entity sets.
4. Identify mapping and participation for each entity set relationships.

ER Diagram of Banking System:

Relational Model:

Software that implements this relation model is called RDBMS.

Each entity set is here table or relation.

Each entity is here tuple or row in that table.

Each attribute is here column of table.

Degree of a table - number of attributes / columns.

Cardinality of table - number of entities / rows.

The values in each table must be atomic - can't be broken further)

Each tuple must be unique.

Keys in Relational Model:

1. Super Key - any subset of attributes which can uniquely identify each tuple.
2. Candidate Key - minimum subset (not null) of super keys which can uniquely identify each tuple.
3. Primary Key - Selected out of candidate keys set which has minimum number of attributes.
4. Alternate Key - all candidate keys except primary key.
5. Foreign Key - primary key of some table (referenced table) used in child table - (referencing table).
6. Compound Key - primary key formed using 2 foreign keys.
7. Composite Key - primary keys formed using at least 2 attributes.
8. Surrogate Key - When tables are merged the primary key of new table is called surrogate key.

Integrity Constraints:

These must be followed for every operation on DB.

1. Domain Constraints - defined data types and range of values of attributes.
2. Entity Constraints - every table / relation should have a primary key.
3. Referential Constraints - follow these rules -
 - Insert Constraint - no value can be inserted in child table if value is not present in parent table.
 - Delete Constraint - tuple in parent can't be deleted until not deleted from the child table. (on delete cascade)
4. Key Constraints - not null, unique, primary key, foreign key, default, check limit or range.

Making relational model from ER model:

1. strong entities become table, weak become table with PK of strong as FK.
2. composite attributes - make them separate columns in table.
3. multi-valued attributes - new tables, PK on entity is FK here.
4. derived attributes - no tables.
5. Generalisation - higher level entity table and lower entity as well - in lower table make column for PK of each higher table.
6. Aggregation - in lower table make column for PK of each higher table.
7. Unary relation - make the relation as table.

Normalisation:

Database normalisation is a design process used to organise tables and columns in a relational database to minimise data redundancy and improve data integrity.

How to implement - Decompose big table to multiple tables, following SRP (Single Responsibility Principle).

Functional Dependency (FD):

A relationship between attributes of table such that if $A \rightarrow B$ means A determines B means if A is known B can also be determined.

Types:

1. *Trivial*: $A \rightarrow B$ where B is a subset of A. ex: {id, name} $\rightarrow$ {name}.
2. *Non-Trivial*: $A \rightarrow B$ where B is not subset of A. ex {id} $\rightarrow$ {name, age, school}

Rules in FD (Armstrong's Axioms):

1. Reflexive - If Y is subset of X then $X \rightarrow Y$ is true.
2. Augmentation - if $X \rightarrow Y$ then $XZ \rightarrow YZ$ is also true.
3. Transitive - if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$ is also true.

Data Redundancy:

Data redundancy means storing the same piece of data in multiple places or systems.

Problems with data redundancy (3 anomalies):

- Due to these consistency issues occur, DB size increase and DB performance also decreases.
1. Insertion anomaly - We can't add data until we don't have all the columns.
 2. Deletion anomaly - If we delete some row, we may lose some data that was not meant to be deleted.
 3. Update anomaly - For a single change, we may get forced to change value in each matching row.

Prime attribute - attribute that is part of PK.

Non Prime attribute - attribute not a part of PK.

Types of Normal Forms:

1. 1-NF - every cell must have atomic values and no multi valued attributes allowed.
2. 2-NF - no partial dependency (**subset** of PK $\rightarrow$ non prime attribute - this should not happen). because suppose if some subset of PK can determine some non prime attribute then its redundancy obviously and it should be separate table only.
3. 3-NF - no transitivity dependency (non prime attribute $\rightarrow$ non prime attribute - this should not happen) because obviously non prime attribute cannot uniquely identify rows so it should not happen.
4. BCNF - If $A \rightarrow B$ then A must be super key. (right side of FD should not be a prime attribute).

ACID Properties & Transactions:

Transaction:

Logical work done by multiple sequential SQL queries whose order matters are illuminated to be an atomic operation using transactions.

If any of the sequential SQL queries fail in a transaction the system is rolled back to the initial state, else the changes get commits at end as an atomic operation.

ACID Properties of transactions:

To ensure the integrity of data, we should follow these properties.

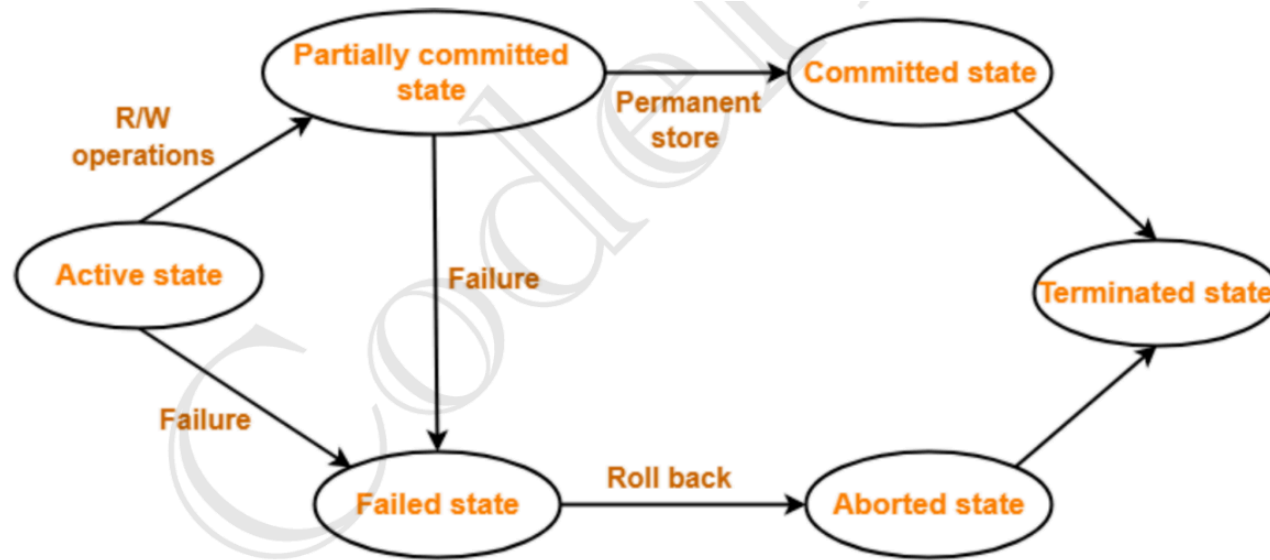
1. Atomicity - either all queries get committed or none gets.
2. Consistency - integrity constraints must be maintained before and after transactions. done by cascade, triggers.
3. Isolation - no two transactions get initiated concurrently (must be done sequentially). done by locks, 4 isolation levels - repeatable reads, read committed, read uncommitted (dirty read problem), serialisable. (2PL locking).
4. Durability - if some transaction has completed then the database now should have persistence.

Keywords in SQL transactions:

Keyword	Purpose
START TRANSACTION / BEGIN	Starts transaction
COMMIT	Permanently saves transaction changes
ROLLBACK	Undoes uncommitted transaction changes
SAVEPOINT	Creates a point to partially roll back to
ROLLBACK TO SAVEPOINT	Undoes changes after that savepoint
SET TRANSACTION	Configures transaction properties, e.g. isolation level
FOR UPDATE	Locks selected rows for update in supported DBs

ND TRIGGER ALSO.

Life Cycle of transaction in DBMS:



Locks vs. Isolation Levels

- **Locks** are the low-level **mechanism** (tools). The database uses them to block or allow access to data rows and tables.
- **Isolation Levels** are the high-level **rules** (policies). They tell the database *how strictly* and *how long* to hold those locks to balance data safety and system speed.

Repeatable Reads

Consistent Reads within same transaction

even if other transaction committed 1st transaction
would not see the changes (if value already read)

Read Committed

Reads within the same transaction always reads fresh value

con: Multiple reads within same transaction are inconsistent

Read Uncommitted

Reads even uncommitted values from other txns

"dirty read"

Serializable

depends on engine



Every read is a locking read

and while one txn read, others will have to wait.

Isolation level	Dirty Read	Non repeatable Read	Phantom
Read Uncommitted	✓	✓	✓
Read Committed	✗	✓	✓
Repeatable Read	✗	✗	✓
Serializable	✗	✗	✗

Phantom Phenomena:

T1 - reads and find x relevant rows

T2 - adds some rows

T1 - finds y number of rows now,
this is called phantom phenomena.

Isolation levels:

The anomalies that we are trying to control from these:

1. Dirty reads: T1 has not committed so T2 sees old data, that's dirty read problem.
2. Non Repeatable Reads: T1 reads, T2 updates, T1 again reads - reads from T1 have different results.
3. Phantom Phenomena: same as 2nd issue but insert/delete rows - rather than updates.

Optimistic and Pessimistic Locking:

Optimistic Locking:

Core idea is that one thread succeeds and other ones fail.

Implemented using comparator and swap atomic function.

Throughput is high and we can configure what to output in failed case.

Pessimistic Locking:

Core idea is one thread at a time, mutually exclusive execution.

Implemented using mutex locks.

High latency, waiting time.

Atomicity Implementation:

1. Recovery Mechanism Component supports atomicity and durability:

- Shadow copy scheme

1. Make a copy first:

When a transaction wants to update the DB, it creates a **complete copy** (new DB). All changes are made on this new copy, while the **old DB remains untouched**.

2. If transaction fails → discard copy:

If the transaction **aborts/fails**, simply delete the new copy. The original DB is still safe.

→ **Either all changes happen or none happen = Atomicity.**

3. If transaction succeeds → switch pointer:

If it succeeds, first make sure the new DB pages are on disk, then update the `db-pointer` to point to the new DB. The new DB becomes the current DB, and the old copy can be deleted.

4. Why it is durable + its drawback:

If the system crashes **before** the pointer changes → old DB is used.

If it crashes **after** the pointer changes → new DB is used.

So committed changes survive crashes (**Durability**).

Drawback: copying the entire DB for every transaction is **very inefficient**.

- Log Based Recovery

1. Keep a log before changing the DB:

Every DB operation is first recorded in a **log stored in stable storage**, and **the log is written before the actual DB change**.

→ If something crashes, we can recover using the log.

2. Deferred DB Modification — write later:

Changes are **only recorded in the log initially**. The actual DB writes happen after the transaction completes.

- Transaction fails/crashes → **ignore the log**
- Transaction succeeds → **perform the deferred writes**
- If failure occurs during writing → **REDO** using the log.

3. Immediate DB Modification — write immediately:

Changes can be written to the DB **while the transaction is still running**. These are called **uncommitted modifications**.

If the transaction fails, we need the **old value** from the log to restore the DB.

4. Recovery rule to remember:

- Transaction fails/aborts → **UNDO** using the **old value**
- Transaction succeeds but system crashes → **REDO** using the **new value**

So the simple memory trick is:

Deferred = REDO

Immediate = UNDO + REDO

Indexing in DBMS:

Indexing is used to optimise the performance of a database by minimising the number of disk accesses required when a query is processed.

3. Indexing Trade-off

Indexes are **not** free.

Advantages

- Faster `SELECT`
- Faster searching
- Faster sorting/range queries for suitable indexes
- Can help enforce uniqueness
- Can speed up joins

Disadvantages

- Extra disk space
- `INSERT` becomes more expensive
- `UPDATE` can become more expensive
- `DELETE` can become more expensive
- Index maintenance overhead
- Too many indexes can hurt write performance

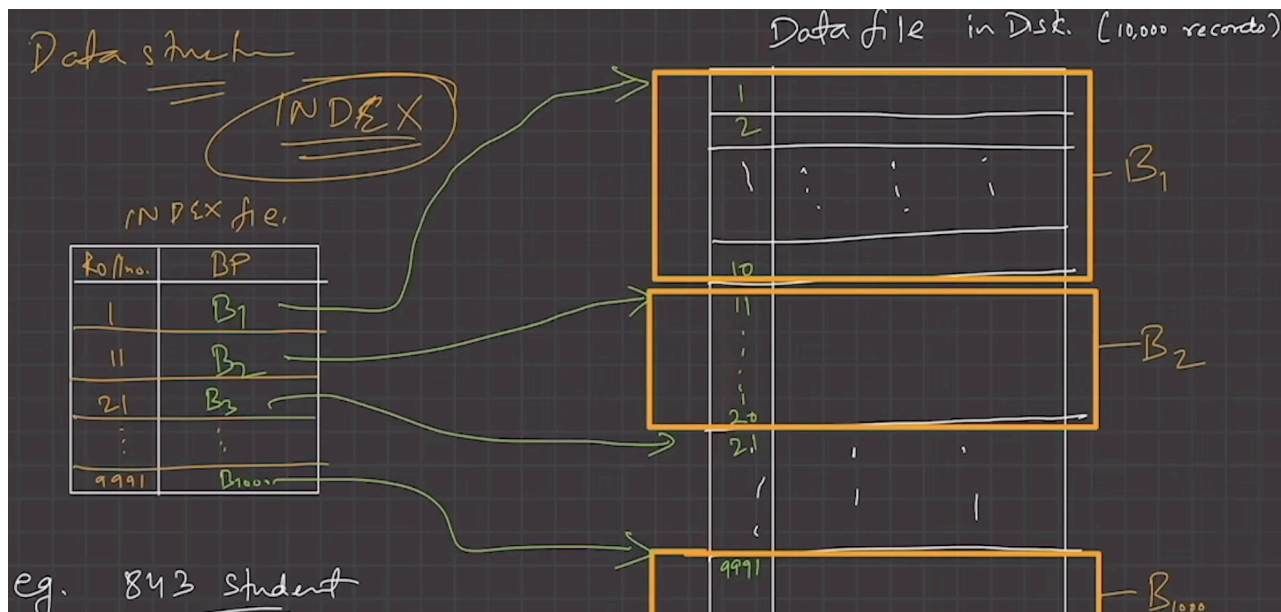
Therefore:

Indexes optimize read performance at the cost of storage and write overhead.

We make blocks of complete table, we now make a new index table and assign base points to first row of each block as primary key will allow us to do this.

It's very easy to understand how we have drastically saved TC for searching on rows.

Index table is always sorted.



</> SQL

```
CREATE INDEX idx_employee_id
ON Employee(employee_id);
```

Dense Indexing - Entry for each row. fast but more overhead.

Sparse Indexing - Entry for first row of each data block (primary key needs to be sorted)

Types of Indexes:

1. Primary Indexing - If any column is sorted we can apply indexing based on that column. often sparse.
 - If table is sorted based on Primary Key then it's called sparse indexing - no need to keep all values.
 - If table is sorted based on non key attribute it's called dense indexing / clustering indexing - have to keep every unique value (determines groups acc to ordering field).
 - when we make an index table for index table, its known as Multi-level indexing
2. Secondary Indexing - an additional index on a field that is not the physical ordering field. (often dense)
3. Ordered Index - Storing indexes in sorted order so that we can apply binary search.

Feature	Clustered	Non-clustered
Data organisation tied to index	Yes	No
Number typically possible	Usually one	Multiple
Range retrieval	Very efficient when aligned	Can require additional lookups
Extra storage	Index is closely integrated with data organisation	Separate index storage
Physical row order	Related to index key	Independent

How is data stored when indexing is done?

Feature	B-Tree	B+ Tree
Data/record pointers	Can exist in internal + leaf nodes	At leaf level
Internal nodes	Keys + pointers	Keys + child pointers
Leaf nodes linked	Not necessarily	Yes
Range queries	Good	Excellent
Fan-out	Generally lower	Generally higher
Common DB indexing	Less common	Very common

Feature	B+ Tree	Hash
Equality search	Excellent	Excellent
Range search	Excellent	Poor
Ordered traversal	Yes	No
Sorting-related use	Yes	No
Average lookup	$O(\log N)$	$O(1)$ average
Prefix/range capabilities	Often useful	Not suitable

Typical use	General-purpose indexing	Equality lookups
-------------	--------------------------	------------------

Other Terms:

These are in system design notes:

Partition

Replication

Sharding

CAP

Master Slave Architecture